

Студент: Чернышева Елизавета Сергеевна
Курс: 3
Группа: Z33433
2025 г.

Лабораторная работа №1

UNIX знакомство: useradd, nano, chmod, docker, GIT, CI, CD

1 Цель работы

Познакомить студента с основами администрирования программных комплексов в ОС семейства UNIX, продемонстрировать особенности виртуализации и контейнеризации, продемонстрировать преимущества использования систем контроля версий (на примере GIT).

2 Задачи

1. [ОС] Работа в ОС, использование файловой системы, прав доступа, исполнение файлов.
 - 1.1 В папке /USR/LOCAL/ создать 2 директории: folder_max, folder_min
 - 1.2 Создать 2-х группы пользователей: group_max, group_min
 - 1.3 Создать 2-х пользователей: user_max_1, user_min_1
 - 1.4 Для пользователей из группы *_max дать полный доступ на директории *_max и *_min. Для пользователей группы *_min дать полный доступ только на директорию *_min
 - 1.5 Создать и исполнить (пользователем из той же категории) скрипт в директории folder_max, который пишет текущую дату/время в файл output.log в текущей директории
 - 1.6 Создать и исполнить (пользователем из той же категории) скрипт в директории folder_max, который пишет текущую дату/время в файл output.log в директории *_min
 - 1.7 Исполнить (пользователем *_min) скрипт в директории folder_max, который пишет текущую дату/время в файл output.log в директории *_min
 - 1.8 Создать и исполнить (пользователем из той же категории) скрипт в директории folder_min, который пишет текущую дату/время в файл output.log в директории *_max
 - 1.9 Вывести перечень прав доступа у папок *_min/ *_max, а также у всего содержимого внутри
2. [КОНТЕЙНЕР] docker build / run / ps / images.
 - 2.1 Создать скрипт, который пишет текущую дату/время в файл output.log в текущей директории

- 2.2 Собрать образ со скриптами выше и с пакетом nano (docker build)
- 2.3 Запустить образ (docker run)
- 2.4 Выполнить скрипт, который подложили при сборке образа
- 2.5 Вывести список пользователей в собранном образе
3. [GIT] GitHub / GitLab, в котором будут содержаться все выполненные ЛР.
 - 3.1 Создать репозиторий в GitHub или GitLab
 - 3.2 Создать структуру репозитория:
 - 3.2.1 lab_01
 - 3.2.1.1 build
 - 3.2.1.2 src
 - 3.2.1.3 doc
 - 3.2.1.4 cmake (для ЛР 1 опционально)
 - 3.2.2 lab_02
 - 3.2.2.1 ... идентично lab_01 ...
 - 3.3 Создать ветки dev / stg / prd, удалить ранее существующие ветки удаленно и локально
 - 3.4 Создать скрипт автоматического переноса ревизий из ветки dev в ветку stg с установкой метки времени (tag). Скрипт в корень репозитория
 - 3.5 Создать скрипт автоматического переноса ревизий из ветки stg в ветку prd с установкой метки времени (tag). Скрипт в корень репозитория
4. [SAVE] Всё, что было сделано в шагах 1-3, сохранить в репозиторий (+ отчет по данной ЛР в папку doc). Фиксацию ревизий производить строго через ветку dev. С помощью скриптов накатить ревизии на stg и на prd.

3 Решение

1. [ОС] Работа в ОС, использование файловой системы, прав доступа, исполнение файлов.
 - 1.1 В папке /USR/LOCAL/ создать 2 директории: folder_max, folder_min:

```
Переходим в корень <cd />.
<cd usr>
<cd local>
<sudo mkdir folder_max>
<sudo mkdir folder_min>
Проверяем, что папки появились: <ls>
```

- 1.2 Создать 2-х группы пользователей: group_max, group_min:

```
<sudo addgroup group_max>
<sudo addgroup group_min>
```

```
Результат:
Adding group 'group_max' (GID 1001) ...
Done
```

Adding group 'group_min' (GID 1002) ...
Done

1.3 Создать 2-х пользователей: user_max_1, user_min_1:

```
<sudo useradd -m -G group_max user_max_1>  
<sudo useradd -m -G group_min user_min_1>
```

Флаг «-m» указывает на создание домашней директории для нового пользователя, флаг «-G» — на добавление пользователя в группу.

Проверяем, что все создалось: `<cat /etc/group>`

Результат:

```
group_max:x:1001:user_max_1  
group_min:x:1002:user_min_1  
user_max_1:x:1003:  
user_min_1:x:1004:
```

1.4 Для пользователей из группы *_max дать полный доступ на директории *_max и *_min. Для пользователей группы *_min дать полный доступ только на директорию *_min:

Установим права доступа к директориям *_max и *_min:

```
<chmod -R 774 folder_max>  
<chmod -R 774 folder_min>
```

Флаг -R означает, что мы устанавливаем права на все файлы внутри директории. Цифры - это права доступа для user, group и others.

4 - чтение (r)

2 - запись (w)

1 - исполнение (x)

Теперь добавим группы к этим директориям:

```
<chown -R :group_max folder_max>  
<chown -R :group_min folder_min>
```

Для проверки текущих прав доступа используем:

`<ls -l>`.

```
total 40  
drwxr-xr-x 2 root root      4096 Sep 11  2024 bin  
drwxr-xr-x 2 root root      4096 Sep 11  2024 etc  
drwxrwxr-- 2 root group_max 4096 Apr 25 08:00 folder_max  
drwxrwxr-- 2 root group_min 4096 Apr 25 08:00 folder_min
```

Чтобы предоставить доступ к директории *_min так же и для *_max группы:

```
<sudo setfacl -Rm g:group_max:rwx folder_min>
```

Для проверки того, добавилась ли группа к директории:

```
<getfacl folder_min>
```

```
# file: folder_min
# owner: root
# group: group_min
user::rwx
group::rwx
group:group_max:rwx
mask::rwx
other::r--
```

- 1.5 Создать и исполнить (пользователем из той же категории) скрипт в директории folder_max, который пишет текущую дату/время в файл output.log в текущей директории

```
<sudo -u user_max_1 bash -c 'echo "date: $(date)» folder_max/output.log'>
```

Проверяем, что все создалось и записалось:

```
<sudo nano folder_max/output.log>
```



Сейчас у нас скрипт создается и выполняется на лету, но чтобы он записался в файл, можно сделать так:

```
<sudo -u user_max_1 bash -c 'cat > folder_max/script_max1.sh <<EOF
#!/bin/bash
echo "date:
$(date)» folder_max/output.log
EOF'>
```

```
<sudo -u user_max_1 bash -c 'chmod +x folder_max/script_max1.sh &&
folder_max/script_max1.sh'>
```

- 1.6 Создать и исполнить (пользователем из той же категории) скрипт в директории `folder_max`, который пишет текущую дату/время в файл `output.log` в директории `*_min`

```
<sudo -u user_max_1 bash -c 'echo "date: $(date)» folder_min/output.log'>
```

Проверяем:



Так же можно записать в файл:

```
<sudo -u user_max_1 bash -c 'cat > folder_max/script_max2.sh <<EOF
#!/bin/bash
echo "date:
$(date)» folder_min/output.log
EOF'>
```

```
<sudo -u user_max_1 bash -c 'chmod +x folder_max/script_max2.sh &&
folder_max/script_max2.sh'>
```

- 1.7 Исполнить (пользователем `*_min`) скрипт в директории `folder_max`, который пишет текущую дату/время в файл `output.log` в директории `*_min`

```
<sudo -u user_min_1 bash -c 'chmod +x folder_max/script_max2.sh &&
folder_max/script_max2.sh'>
```

Так как у `*_min` пользователя нет прав доступа к директории `*_max`, должна возникнуть ошибка:

```
chmod: cannot access 'folder_max/script_max2.sh': Permission denied
```

- 1.8 Создать и исполнить (пользователем из той же категории) скрипт в директории `folder_min`, который пишет текущую дату/время в файл `output.log` в директории `*_max`

```
<sudo -u user_min_1 bash -c 'cat > folder_min/script_min1.sh <<EOF
```

```
#!/bin/bash
echo "date:
$(date)» folder_max/output.log
EOF">
```

```
<sudo -u user_min_1 bash -c 'chmod +x folder_min/script_min1.sh &&
folder_min/script_min1.sh'>
```

Так как доступа к директории *_max у пользователя *_min нет, получаем ошибку:

```
folder_min/script_min1.sh: line 2: folder_max/output.log: Permission denied
```

1.9 Вывести перечень прав доступа у папок *_min/ *_max, а также у всего содержимого внутри

Права и содержимое folder_max:

<ls -ld folder_max> - вывод информации о директории (права, владелец, группа, размер, дата изменения); флаг -d позволяет вывести информацию о самой папке, а не ее содержимом.

<sudo ls folder_max> - позволяет вывести содержимое директории.

<getfacl folder_max> - показывает пользователя, группы, а так же права всех категорий пользователей к этой директории.

```
<cat folder_max/output.log>
<cat folder_max/script_max1>
<cat folder_max/script_max2>
cat выводит содержимое файла.
```

```
ubuntu@elizaveta:/usr/local$ ls -ld folder_max
drwxrwxr-- 2 root group_max 4096 Apr 25 16:37 folder_max
ubuntu@elizaveta:/usr/local$ sudo ls folder_max
output.log  script_max1.sh  script_max2.sh
ubuntu@elizaveta:/usr/local$ getfacl folder_max
# file: folder_max
# owner: root
# group: group_max
user::rwx
group::rwx
other::r--
```

```
ubuntu@elizaveta:/usr/local$ sudo cat folder_max/output.log
date: Fri Apr 25 04:33:33 PM UTC 2025
ubuntu@elizaveta:/usr/local$ sudo cat folder_max/script_max1.sh
#!/bin/bash
echo "date: $(date)" > folder_max/output.log
ubuntu@elizaveta:/usr/local$ sudo cat folder_max/script_max2.sh
#!/bin/bash
echo "date: $(date)" > folder_min/output.log
ubuntu@elizaveta:/usr/local$
```

Аналогично для директории folder_min:

```

ubuntu@elizaveta:/usr/local$ ls -ld folder_min
drwxrwxr--+ 2 root group_min 4096 Apr 25 17:04 folder_min
ubuntu@elizaveta:/usr/local$ sudo ls folder_min
output.log  script_min1.sh
ubuntu@elizaveta:/usr/local$ getfacl folder_min
# file: folder_min
# owner: root
# group: group_min
user::rwx
group::rwx
group:group_max:rwx
mask::rwx
other::r--

ubuntu@elizaveta:/usr/local$ sudo cat folder_min/output.log
date: Fri Apr 25 04:37:39 PM UTC 2025
ubuntu@elizaveta:/usr/local$ sudo cat folder_min/script_min1.sh
#!/bin/bash
echo "date: $(date)" > folder_max/output.log
ubuntu@elizaveta:/usr/local$

```

2. [КОНТЕЙНЕР] docker build / run / ps / images.

2.1 Создать скрипт, который пишет текущую дату/время в файл output.log в текущей директории

Создаем папку с проектом, куда добавляем скрипт файл script.sh

```

● chernysheva@adminacc:~/cpp_lab_1$ ls
Dockerfile  script.sh

```

```

GNU nano 6.2 script.sh
#!/bin/sh
echo "Current datetime: $(date)" > output.log

^G Справка      ^O Записать     ^W Поиск       ^K Вырезать    ^T Выполнить   ^C Позиция     ^U Отмена
^X Выход        ^R ЧитФайл     ^E Замена      ^V Вставить    ^_ Выровнять   ^_ К строке   ^-Е Повтор

```

2.2 Собрать образ со скриптами выше и с пакетом nano (docker build)

Создаем в папке с проектом Dockerfile:

```

Dockerfile ×
Dockerfile > ...
1 FROM alpine:latest
2 RUN apk add nano
3 COPY ./script.sh /scripts/date_time.sh
4 WORKDIR /scripts
5 CMD ["/bin/sh"]

```

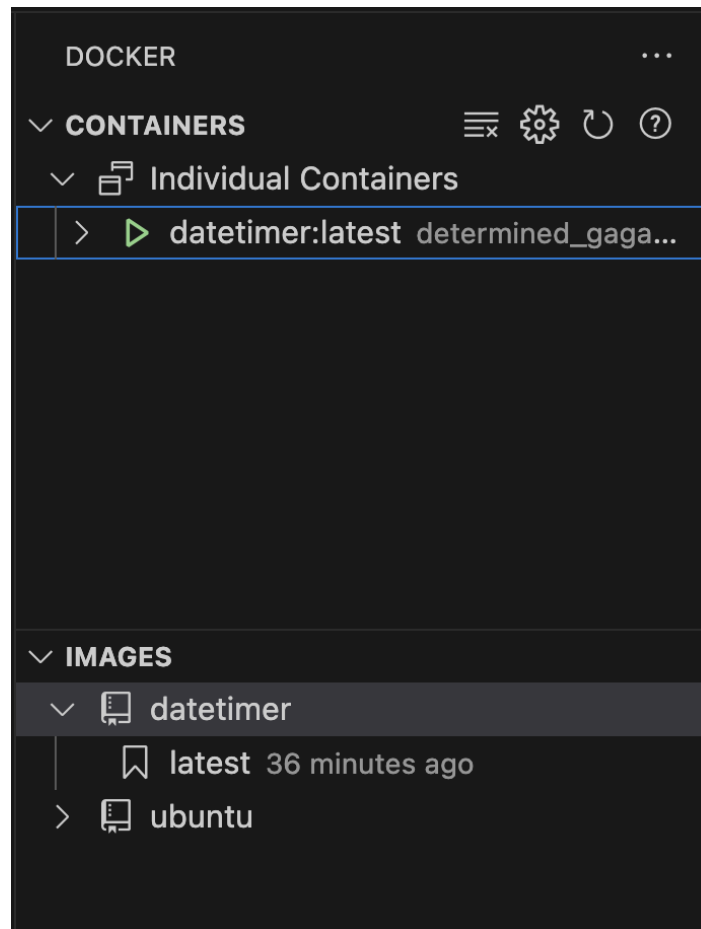
Здесь в первой строке мы прописываем дистрибутив линукс, образ которого хотим установить с помощью докера. Допустим, легковесный alpine. Во второй строке прописываем нужную нам утилиту командной строки линукса, здесь это nano. В третьей строке мы копируем наш скрипт файл в создаваемый образ в папку scripts и называем его data_time.sh. В четвертой строке мы прописываем рабочую директорию - как только контейнер будет запущен, мы сразу окажемся в ней. И последняя строка - оболочка для

возможности использования линуксовских команд alpine в созданном образе.

2.3 Запустить образ (docker run)

`<docker build -t datetime_image .>` - устанавливаем образ, согласно заданным критериям в Dockerfile. В конце команды указывается путь к докер файлу, поэтому в указанной директории он должен быть только один. Флаг `-t` позволяет задать имя.

`<docker run -it --name datetime_container datetime_image>` - для запуска контейнера на основании образа. Флаг `-it` позволяет открыть контейнер в интерактивном режиме.



2.4 Выполнить скрипт, который подложили при сборке образа

```
/scripts # ls
date_time.sh
/scripts # ./date_time.sh
/scripts # ls
date_time.sh output.log
/scripts # nano output.log
```

2.5 Вывести список пользователей в собранном образе

`<cat /etc/passwd>`

root - основной суперпользователь,
остальные записи (bin, daemon и т.д.) — системные пользователи (не для входа).


```
GNU nano 8.2 output.log
Current datetime: Fri Apr 25 19:43:21 UTC 2025

[ Read 1 Line ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^I Execute    ^G Location   ^M-U Undo
^X Exit      ^R Read File  ^F Replace    ^U Paste      ^J Justify    ^_ Go To Line  ^M-E Redo
```

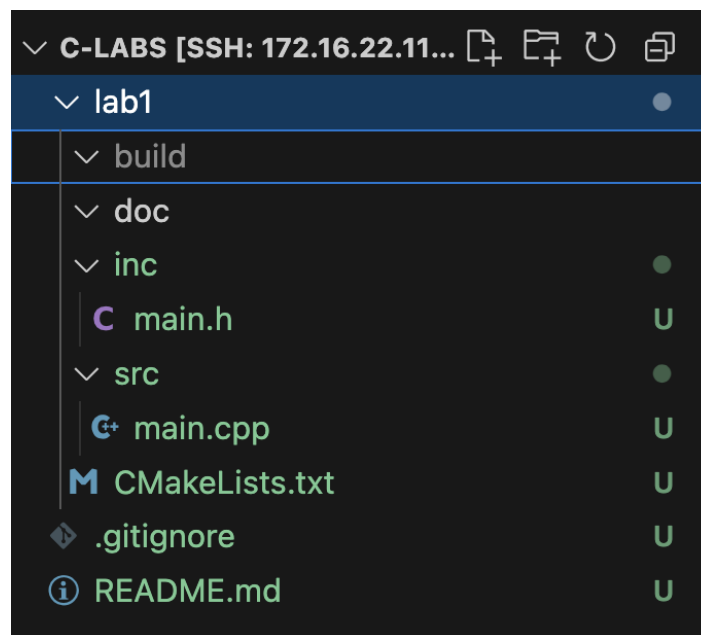
```
/scripts # cat /etc/passwd
root:x:0:0:root:/root:/bin/sh
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/mail:/sbin/nologin
news:x:9:13:news:/usr/lib/news:/sbin/nologin
uucp:x:10:14:uucp:/var/spool/uucppublic:/sbin/nologin
cron:x:16:16:cron:/var/spool/cron:/sbin/nologin
ftp:x:21:21:ftp:/var/lib/ftp:/sbin/nologin
sshd:x:22:22:sshd:/dev/null:/sbin/nologin
games:x:35:35:games:/usr/games:/sbin/nologin
ntp:x:123:123:NTP:/var/empty:/sbin/nologin
guest:x:405:100:guest:/dev/null:/sbin/nologin
nobody:x:65534:65534:nobody:/:/sbin/nologin
```

3. [GIT] GitHub / GitLab, в котором будут содержаться все выполненные ЛР.

3.1 Создать репозиторий в GitHub или GitLab

<https://github.com/Elizabeth3344/C-labs.git>

3.2 Создать структуру репозитория



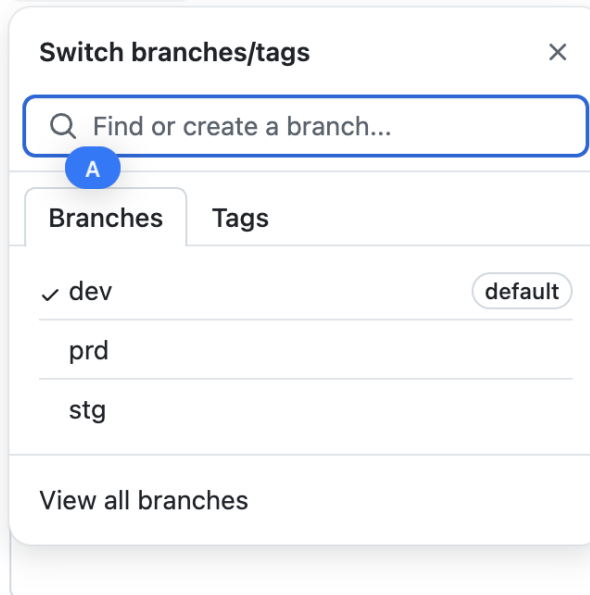
3.3 Создать ветки dev / stg / prd, удалить ранее существующие ветки удаленно и локально

dev - ветка для разработки проекта

stg - ветка для тестирования

prd - ветка для рабочего проекта

dev 3 Branches 0 Tags



3.4 Создать скрипт автоматического переноса ревизий из ветки dev в ветку stg с установкой метки времени (tag). Скрипт в корень репозитория

в корне репозитория создаем файл stage.sh

```
$ stage.sh U X
$ stage.sh
1 git checkout stg
2 git merge dev
3 timetag=$(date '+%H-%M-%S')
4 git tag $timetag
5 git checkout dev
```

Перед выполнением файла делаем:

<sudo chmod +x stage.sh> - для передачи прав доступа к файлу. Флаг +x добавляет права на исполнение файла. После этого можно пушить изменения в гитхаб.

```

● chernysheva@adminacc:~/C-labs$ ./stage.sh
Переключились на ветку «stg»
Эта ветка соответствует «origin/stg».
Обновление 2d137ad..103e639
Fast-forward
 stage.sh | 7 ++++++
 1 file changed, 7 insertions(+)
 create mode 100755 stage.sh
Переключились на ветку «dev»
Эта ветка соответствует «origin/dev».

```

```

● chernysheva@adminacc:~/C-labs$ git show-ref --tags -d
28105fa9e2b0db41a5b9bf730352b09f96b4aca3 refs/tags/00-00-09
103e6398a905b257984be78239e0c43af4661fd3 refs/tags/00-02-21

```

3.5 Создать скрипт автоматического переноса ревизий из ветки stg в ветку prd с установкой метки времени (tag). Скрипт в корень репозитория

Все аналогично предыдущему пункту. Создаем файл prod.sh. Не забываем про права доступа к файлу.

```

$ prod.sh U ×

$ prod.sh
1  #!/bin/bash
2
3  git checkout prd
4  git merge stg
5  timetag=$(date '+%H-%M-%S')
6  git tag $timetag
7  git checkout dev

```

```

● chernysheva@adminacc:~/C-labs$ ./prod.sh
Переключились на ветку «prd»
Эта ветка соответствует «origin/prd».
Обновление 2d137ad..103e639
Fast-forward
 stage.sh | 7 ++++++
 1 file changed, 7 insertions(+)
 create mode 100755 stage.sh
Переключились на ветку «dev»
Эта ветка соответствует «origin/dev».
● chernysheva@adminacc:~/C-labs$ git show-ref --tags -d
28105fa9e2b0db41a5b9bf730352b09f96b4aca3 refs/tags/00-00-09
103e6398a905b257984be78239e0c43af4661fd3 refs/tags/00-02-21
103e6398a905b257984be78239e0c43af4661fd3 refs/tags/00-15-28
○ chernysheva@adminacc:~/C-labs$

```

4. [SAVE] Всё, что было сделано в шагах 1-3, сохранить в репозиторий (+ отчет по данной ЛР в папку doc). Фиксацию ревизий производить строго через ветку dev. С помощью скриптов накатить ревизии на stg и на prd.